ヨットにおける最適解を求めるコードの実装

門倉春介

2026 年 5 月 11 日

1　はじめに

※生成 AI 注意※

この研究資料の内容のほとんどはとあるヨットの最適解を求めるアルゴリズムを分かりやすく説明したサイトの内容を読んで、アルゴリズムをできるだけ数学チックにまとめたものである。

そして、このアルゴリズムを Python を用いてコーディングした（長いのでコードは省略）。

当初は Python に慣れるためにこの研究を始めたので AI は使っていなかったが、コードが非効率で計算に膨大な時間がかかったため、途中から生成 AI にコードを書かせた。

2　ヨットのルール

本研究では、Nintendo Switch 用ソフト『世界のアソビ大全 51』内のゲーム「ヨット[1]」に準拠したルールを採用する。ヨットは 5 つのダイスを振り、役を作って得点を競うゲームである。

2.1　ゲームの進行

1. 1 回の番につき、ダイスを最大 3 回まで振ることができる。
2. 1 回目、2 回目に振った後、任意の個数のダイスを「キープ（確保）」し、残りを振り直すことができる。
3. 最終的な 5 つのダイスの目を用いて、スコアボードの空いている項目のいずれかに点数を記録する。
4. 全 12 項目が埋まった時点の合計得点を最終スコアとする。

2.2　スコア項目とボーナス

得点計算は以下の表に従う。特に、上段（1〜6 の目）の合計が 63 点以上の場合、35 点のボーナスが加算される点が戦略上重要である。

表 1: ヨット スコア表（アソビ大全準拠）

項目名	説明	得点例
1 〜 6	指定された目の合計	「1」の目が 3 つの場合：3 点
小計・ボーナス	**上記 6 項目の合計が 63 点以上**	**35 点加算**
チョイス	全ダイスの合計	任意の合計値
フォーダイス	同じ目が 4 つ以上	全ダイスの合計
フルハウス	3 つ同じ目 ＋ 2 つ同じ目	全ダイスの合計
S ストレート	4 つの目が連続	15 点固定
B ストレート	5 つの目が連続	30 点固定
ヨット	5 つ全て同じ目	50 点固定

[1]数理科学研究会で一時期流行っていたと言われている。

3　ヨットの最適解を求めるアルゴリズム

3.1　用語の導入

勉強したサイトから用語を流用。

出目パターン　5 個のダイスの組み合わせ。ダイスの順序は無視するため、昇順に並べた 5 つ組の集合 $S \in \{(d_1, d_2, d_3, d_4, d_5) \mid 1 \leq d_1 \leq d_2 \leq \cdots \leq d_5 \leq 6\}$ と定義する。

部分出目パターン　k 個 $(0 \leq k \leq 5)$ のダイスの組み合わせ。キープされたダイスの状態を示す。

含まれる　ある部分出目パターン p が、出目パターン P に含まれるとは、P の要素からうまく k 個選ぶことで p を構成できることを指す。

- 例：部分出目パターン $(1, 5, 6)$ は 出目パターン $(1, 1, 3, 5, 6)$ に含まれる。
- 空集合（0 個の部分出目パターン）は全ての出目パターンに含まれる。

3.2　最適戦略の導出手順

まず、12 ターン目の最後の振り直しをして獲得できる点数を最大化する方法から考える。

3.2.1　手順 1：遷移確率の事前計算

ある k 個からなる部分出目パターン p に対し、残りの $5 - k$ 個のダイスを振って、最終的に出目パターン P になる確率 $Pr(P \mid p)$ を算出する。すべての (p, P) の組み合わせについてテーブル化しておく。

3.2.2　手順 2：役得点計算の事前計算

各出目パターン P に対し、各スコア項目 $c \in \{1 \ldots 12\}$ に当てはめた際の得点 $Score(P, c)$ を算出する。

$$\mathrm{Table}_{score}[P][c] = \text{得点値} \tag{1}$$

3.2.3　手順 3：残り 1 回の振り直し時点（$k = 1$）の最大期待値

2 回目のダイスを振り終えた直後の出目パターンを P_1 とする。このとき、プレイヤーは P_1 に含まれる任意の「部分出目パターン p」をキープし、残りのダイスを振り直す。

1. キープ p に対する期待値の計算:

部分出目パターン p を選択したとき、最終的な出目パターン P_0 に遷移する確率を $Pr(P_0 \mid p)$ とすると、その期待値 $E(p, c)$ は次式で表される。

$$E(p, c) = \sum_{P_0} Pr(P_0 \mid p) \times \mathrm{Score}(P_0, c) \tag{2}$$

2. 最適なキープの選択

現在の出目 P_1 において、期待値を最大化するキープ p^* を選択する。このときの最大期待値を $V_1(P_1, c)$ と定義し、テーブル化する。

$$V_1(P_1, c) = \max_{p \subseteq P_1} E(p, c) \tag{3}$$

3.2.4　手順 4：残り 2 回の振り直し時点（$k = 2$）の最大期待値

1 回目のダイスを振り終えた直後の出目パターンを P_2 とする。手順 3 と同様に、ここでも最適なキープを選択する。

1. キープに対する期待値計算:

部分出目パターン p をキープして振り直した結果、次に現れる出目パターンを P_1 とする。この P_1 は手順 3 における「開始状態」に相当するため、既に最大期待値 $V_1(P_1, c)$ が計算済みである。したがって、期待値 $E(p, c)$ は次のように求められる。

$$E(p, c) = \sum_{P_1} Pr(P_1 \mid p) \times V_1(P_1, c) \tag{4}$$

2. 最大期待値のテーブル化:
P_2 からの最適戦略による最大期待値を $V_2(P_2, c)$ として保存する。

$$V_2(P_2, c) = \max_{p \subseteq P_2} E(p, c) \tag{5}$$

3.2.5　手順 5：ターン開始時の最終期待値の算出

ダイスを一度も振っていない状態（$k = 3$）から、その役 c を選んだ場合の最終的な期待値 $V_3(c)$ を求める。
1. 一投目の確率:
最初に 5 個のダイスを振った際、出目パターン P_2 が出現する確率を $Pr(P_2)$ とする。
2. 期待値の総和: 手順 4 で求めた $V_2(P_2, c)$ に $Pr(P_2)$ をかけ、全ての P_2 について和をとることで、役 c におけるターン開始時の期待値が求まる。

$$V_3(c) = \sum_{P_2} Pr(P_2) \times V_2(P_2, c) \tag{6}$$

これを全ての役に対して計算する。

3.2.6　手順 6：ボーナスの考慮

1. 状態変数の定義:
最終ターン開始時点の状態を $S = \{c_{last}, s_{upper}, b\}$ と定義する。

- c_{last}：最後に残っている役
- s_{upper}：現在の第 1〜6 の役の小計 $(0 \leq s_{upper} \leq 105)$
- b：ボーナス（35 点）を既に獲得済みかどうかのフラグ

2. ボーナスを内包した得点関数:
出目パターン P を役 c_{last} に適用した際の得点 $Score^*(P, c_{last}, s_{upper}, b)$ は以下のように定義される。

$$Score^*(P, c, s, b) = \begin{cases} Score(P, c) + 35 & (\text{if } b = \text{false},\ c \in \text{UpperHands, and } s + Score(P, c) \geq 63) \\ Score(P, c) & (\text{otherwise}) \end{cases} \tag{7}$$

この得点関数を用いることでボーナスを考慮した 12 ターン目の期待値 $V_{final}(c_{last}, s_{upper}, b)$ が求まる。

4　ゲーム全体の最適解を求める

前章までで、最終ターン（12 ターン目）における各状況下の最大期待値を算出した。本章ではこれを基底（ベースケース）とし、時間を遡ることでゲーム開始時点（1 ターン目）における最大期待値を導出する手法について述べる。

4.1

n ターン目において、ある出目 P が確定したときを考える。このとき最適な行動は、単にそのターンでの得点を最大化することではなく、「このターンでの得点」と「残りの役で次ターン以降（$n+1$ ターン目以降）に得られる最大期待値」の和を最大化することである。

具体例として、残り 2 手（11 ターン目）の状況を考える。現在の空き役の集合を $\mathcal{C}_{open}$、現在の小計を s とする。ある役 $c \in \mathcal{C}_{open}$ を選択し、その得点が $score$ であった場合、次ターン（12 ターン目）へ引き継がれる状態は以下のようになる。

- 次ターンの空き役：$\mathcal{C}' = \mathcal{C}_{open} \setminus \{c\}$
- 次ターンの小計：$s' = s + (\text{if } c \in \text{UpperHands then } score \text{ else } 0)$

このとき、役 c を選んだ際の合計期待値 E_{total} は次式で定義される。

$$E_{total}(c, s, P) = score(P, c) + V_{n+1}(\mathcal{C}', s') \tag{8}$$

ここで V_{n+1} は、次ターン以降の全手番を最適にプレイした際の最大期待値であり、事前に計算済みである必要がある。

4.2

以上のプロセスを、残り手番数 $N = 12$ から 1 まで繰り返す。

1. **状態空間の構築**: 各ターン n において、「残っている役の組み合わせ（$\binom{12}{12-n+1}$ 通り）」と「上段小計（0〜63 点以上）」のすべてのパターンについて、期待値テーブルを埋めていく。
2. n ターン目のテーブルを埋めるには、$n+1$ ターン目のテーブルを参照する。これを 12 ターン目から 1 ターン目まで後退しながらやっていく。
3. **ゲーム開始時の期待値**: 最終的に 1 ターン目（残り役 12 個、小計 0 点）の計算結果 $V_1(\text{役全部}, 0)$ が、導き出されたヨットの最大期待値となる。

   ちなみにこのようにして計算結果をその都度メモ化して参照しながら効率的に計算するのを、アルゴリズムで動的計画法というらしい。

5　実戦において最適なキープをその都度教えてくれる機械

前章までのアルゴリズムにより、ゲーム開始時点からの最大期待値を導出することができた。そしてデータ上、この最大期待値という数字だけでなく、実際にどんな状況でどんなキープをするのが最適かということまで分かるはずである。

データの抽出

紹介してきたアルゴリズムの副産物として、期待値 V を算出する過程で、各状態における最適な行動を同時に特定できる。具体的には、以下の 3 つの要素が決定された瞬間に、最適なキープや役の選択がデータとして確定する。

- 現在のターン数および空き役の集合（$\mathcal{C}_{open}$）
- 現在の上段小計（s_{upper}）
- 現在の出目パターンと残り振り直し回数（k）

これらの条件下で期待値を最大化する選択（どのダイスをキープするか、あるいはどの役を埋めるか）をメモする。

一度計算してメモしてしまえばそれを参照するだけで良いので、コンピュータならば一瞬でやってくれる。

これで、ユーザが現在の状況（ダイスの目、残り役、小計）を入力するだけで、計算済みのメモを参照し、即座に「最適なキープ」や「埋めるべき役」を表示するコンピュータができる。

6　まとめ

本当はこの最適解を求めるコードをたくさん実戦形式で自動周回させてデータを集めたのだが、出たデータから戦略が上手く読み取れなかったので没となった。

おそらく

- なるべく多くの役を同時に狙いうるキープをしたい
- ACES や YACHT はこういう状況なら捨てたい

みたいなことをコンピュータも考えているのではないだろうか。少し自分でも遊んでみて思ったのは、２個でも連続しているのがあるとそれが選ばれがちということ。

7　参考資料

サイト：

【世界のアソビ大全 51】ヨットを数学的に考える 1：基礎知識編
【世界のアソビ大全 51】ヨットを数学的に考える 2：役の期待値を最大化する
【世界のアソビ大全 51】ヨットを数学的に考え 3：合計点数の期待値を最大化する